

Procédure de Déploiement Flutter

Guide complet de préparation et publication en Release (Android & iOS)

Ce document détaille les étapes nécessaires pour préparer, signer et générer les exécutables de production (Release) pour les plateformes Android (Google Play Store) et iOS (App Store / TestFlight).

■ PARTIE 1 : Déploiement Android (Google Play Store)

Étape 1 : Générer la clé de signature (Keystore .jks)

Exécutez la commande suivante dans le terminal pour générer la clé de signature de votre application :

```
keytool -genkey -v -keystore mon-app-key.jks -keyalg RSA -keysize 2048 -validity 10000 -alias release
```

■■ **IMPORTANT** : Conservez précieusement votre fichier .jks, son mot de passe et son alias dans un endroit sécurisé (gestionnaire de mots de passe, cloud sécurisé). En cas de perte de cette clé, il sera **strictement impossible** de publier des mises à jour de votre application sur le Google Play Store !

Étape 2 : Configurer la signature dans le projet

1. **Placer le Keystore** : Placez votre fichier mon-app-key.jks dans le dossier android/ ou à la racine du projet.

2. **Créer android/key.properties** : Créez ce fichier et renseignez-y vos identifiants :

```
storePassword=VotreMotDePasseSecurise
keyPassword=VotreMotDePasseSecurise
keyAlias=release
storeFile=../mon-app-key.jks
```

3. **Configurer android/app/build.gradle.kts (Kotlin DSL)** :

```
import java.util.Properties
import java.io.FileInputStream

val keyPropertiesFile = rootProject.file("key.properties")
val keyProperties = Properties()
if (keyPropertiesFile.exists()) {
    keyProperties.load(FileInputStream(keyPropertiesFile))
}

android {
    ...
    signingConfigs {
        create("release") {
            keyAlias      = keyProperties["keyAlias"] as String
            keyPassword   = keyProperties["keyPassword"] as String
            storeFile     = file(keyProperties["storeFile"] as String)
            storePassword = keyProperties["storePassword"] as String
        }
    }

    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
            isMinifyEnabled = false
        }
    }
}
```

```
        isShrinkResources = false
    }
}
}
```

Étape 3 : Gestion de la version (pubspec.yaml)

À chaque nouveau déploiement, vous devez incrémenter le numéro de version dans pubspec.yaml :

version: 1.0.0+1

- 1.0.0 : **versionName** (numéro de version affiché aux utilisateurs sur le store).
- +1 : **versionCode** (numéro de build interne, obligatoirement incrémenté à chaque envoi : +1, +2, +3...).

Étape 4 : Générer l'Android App Bundle (.aab)

Exécutez les commandes suivantes dans votre terminal :

```
flutter clean
flutter pub get
flutter build appbundle --release
```

Le fichier prêt pour le Play Store sera généré dans :
build/app/outputs/bundle/release/app-release.aab

■ PARTIE 2 : Déploiement iOS (App Store / TestFlight)

Étape 1 : Configuration dans Xcode

1. Ouvrez le projet dans Xcode avec la commande : open ios/Runner.xcworkspace
2. Sélectionnez la cible **Runner** > Onglet **Signing & Capabilities**.
3. Sélectionnez votre ****Team**** Apple Developer et vérifiez le ****Bundle Identifier****.

Étape 2 : Build et Publication de l'Archive

Dans votre terminal, générez le paquet iOS :

```
flutter build ipa --release
```

Alternativement depuis Xcode : menu **Product > Archive** puis **Distribute App** vers App Store Connect.

■ Checklist Récapitulative de Déploiement

- [] Code testé et validé sans bugs majeurs
- [] Version incrémentée dans pubspec.yaml (ex: 1.0.0+2)
- [] Fichier key.properties et Keystore .jks configurés
- [] Fichier google-services.json (Android) / GoogleService-Info.plist (iOS) présents si Firebase est utilisé
- [] Commande flutter build appbundle --release exécutée avec succès

[] Fichier app-release.aab téléversé sur la Google Play Console

[] Sauvegarde du Keystore et des mots de passe effectuée en lieu sûr